Characterizing Software and Hardware Coherence for Synchronization in CXL Fabric-Attached Memory

Anonymous Author(s)
Submission Id: 806

Abstract

The rise of CXL 3.0 fabric-attached memory enables multiple hosts to share memory transparently, creating new opportunities for distributed AI and data analytics. Yet efficient synchronization across hosts remains a core challenge. We present the first comprehensive study of mutex performance across CXL's two coherence modes: (1) software-managed coherence using load/store operations with fences, and (2) hardware-managed coherence using read-modify-write atomics with back-invalidation snooping. Using a BittWare IA-780i FPGA platform, we implement the first hardware-coherence prototype with programmable delay injection, allowing us to emulate memory latencies from hundreds of nanoseconds to tens of microseconds. Our evaluation spans classical software locks (e.g., Bakery, Peterson, Elevator) and hardware locks (e.g., spin, ticket, MCS). Results reveal sharp trade-offs: hardware locks excel at low contention and short latencies but collapse under heavy cache-line contention, while software locks scale predictably and remain competitive in high-latency settings despite higher baseline costs. We identify crossover points where software synchronization outperforms hardware coherence, showing that full cache-coherence support is not always necessary. These findings provide guidance for both system designers and developers optimizing synchronization for disaggregated CXL deployments.

1 Introduction

The advent of AI and LLM driven workloads increasingly requires datacenter platforms to provide high bandwidth networks and support for dedicated accelerators. This computational and memory pressure is coupled with user demands for lower latency, as AI server-side applications are increasingly exposed to end users and novel domains with tight SLO considerations.

A possible programming model that can support these novel workloads is a new iteration of an old idea, that of collaborative processing over shared data across server boundaries. Distributed shared memory has long been of interest to the systems community as it provides a fast mechanism to shared data without incurring network overheads in the form of serialization and data copies[41]. Recent advances in datacenter interconnects enable such an architecture with scalable fabric attached memory (FAM). The Compute Express Link (CXL) open standard specifies supports for a multi-host, multi-device, switchable architectures for shared FAM. This device topology enables multiple processes in different hosts to transparently collaborate on workloads on a shared address space. Shared memory collaboration offers the opportunity to reduce data movement overheads across the datacenter, potentially providing a lower overhead platform for heterogeneous workloads over large scale data.

Why?

The key to enabling a shared-memory programming environment at scale is to provide efficient methods for concurrent processes to synchronize their accesses to a shared-address space. A fundamental synchronization mechanism is the humble mutex/lock. Mutexes synchronize concurrent threads memory accesses by ensuring that only one thread can enter a "critical section" at a time. This ensures an ordering of memory accesses, allowing concurrent threads to collaborate over shared data without causing any data races that undermine the correctness of the application.

Is that all? why is this the key?

The underlying efficiency of a mutex has significant performance implications towards any concurrent application that relies on them for synchronization. This coupling has resulted in significant iteration and innovation around developing efficient mutex implementations for different memory environments over the years. The underlying characteristics of a given memory system , in the form of it's consistency model, cache coherence protocol, and availability of concurrency primitives, determine which mutex implementation is the best fit for a given environment. CXL's Shared FAM in particular offers two distinct memory coherence models in multi host environment; software-managed coherence and hardware coherence. These two coherence models motivate different mutex implementations with their own performance characteristics.

Not sure why you are telling us this.

A unique proposition of CXL 3.X's specification is that of multi-host switchable hardware cache coherence. This provides a shared address space with the capacity of seamlessly managing host cache lines without requiring host to host communication. This enables not only significant latency benefits for applications (as they can now leverage their cache hierarchy for lower latency access) but also crucially for synchronization the use of read-modify-write(rmw) atomic instructions. CXL provides supports for these complex atomics by leveraging hardware cache coherence to grant exclusive cache line ownership to a given host, thus permitting them to execute their own read-modify-write atomic instructions within their silo. This capability permits synchronization primitives broadly to leverage powerful hardware concurrency primitives that have traditionally

supported the most efficient of mutex implementations. This permits the implementation of "hardware locks", which have generally been the most efficient at high contention. This comes at the corresponding cost of supporting the hardware cache coherence protocol at scale, which up to now has remained unexplored in terms of performance analysis.

Seems like you should provide a reference.

Software-managed cache coherence as outlined by the CXL 3.X's specification requires the software layer to manage copies of shared state explicitly through the use of load/store operations and fences. This is sufficient to achieve mutual exclusion through the use of "software locks". This category of mutexes do not rely on any read-modify-write atomics, but instead rely on software control structures and load/store instructions to synchronize. In the case of CXL, this obviates the need of relying on the hardware cache coherence protocol entirely.

These two methods to implement a mutex and thus provide synchronization for processes collaborating in a shared FAM environment rely on distinct mechanisms with their own performance implications. Understanding their performance characteristics is key to deciding not only which mutex implementation is useful for a given application but whether certain applications can avoid relying on full multi-host hardware coherence to synchronize.

OK

This paper makes the following contributions:

What have we learned?

1. Implementation of an open source mutex benchmark targeting a CXL Shared FAM environment[36].
2. The first implementation of CXL hardware coherence with back-invalidation on an FPGA.
3. An evaluation of mutex synchronization relying on software and hardware coherence as specified by the CXL specification.

2 Background

The following section provides an overview of why shared FAM memory can address the datacenter challenges of AI applications, the programming model that CXL enables and the requirements to support it, the key characteristics of the cache coherence protocols supported by CXL, and an introduction to software and hardware locks and their shared characteristics.

2.1 AI in the Datacenter

Artificial Intelligence(AI) applications are becoming increasingly prevalent in the era of Large Language Models (LLMs). Distributed training for AI in particular is straining existing datacenter network resources, with existing large scale systems struggling to fully utilize their existing compute due to network overheads and bandwidth limitations[37][42][33]. These scaling challenges are exacerbated by existing Two-tier network architectures in AI training. These consist of GPU clusters connected over a high-bandwidth interconnect such as NVLink for intra node traffic while carrying out inter-node traffic with CPUs over lower bandwidth Ethernet networks[42][14]. Two tier networks increase communication overhead when communicating between GPU clusters and create an environement of uneven communication bandwidth between CPUs and GPUs. With increasing interest in AI workloads hybridizing training and inferencing with heterogeneous compute, unequal bandwidth between different kinds of processors stands as an obstacle for novel hybridized workloads[34].

Compared to what?

The datacenter must adapt to respond to the new application paradigm and provide substantial heterogeneous compute capacity to support hybrid workloads, high bandwidth memory to address the ever increasing magnitude of data, and low-latency networks to efficiently saturate resources.

There has been notable success in providing such a datacenter architecture with Remote Direct Access Memory (RDMA)[14]. However of deployment of RDMA at scale generally takes the form of RDMA over Converged Ethernet (RoCE), which seems ill poised to scale to the ever increasing demands of AI applications [19]. Why?

CXL stands to be an ideal platform for enabling a shared memory environment that reduces network overheads, and provides low latency access and high bandwidth for heterogeneous processing.

Key to enabling this vision is providing efficient implementations of ubiquitous communication primitives seen in AI workloads such as AlltoAll, AllReduce, AllGather, and ReduceScatter [14]. The implementations of these parallel communication primitives in a manner congruent with multithreaded applications requires thread safety for certain shared data structures by mechanisms that ensure inter-thread synchronization and mutual exclusion [9][4][12]. This requires the implementation of efficient inter-node locks. The scalability of such locks is key to the performance of shared data structures, as non scalable locks can cause dramatic performance collapse in real workloads [5]. Locks remain a foundational primitive to enable efficient multi process collaboration in a large scale shared memory platform, justifying their study in the novel CXL environment.

2.2 Memory Architecture

Following the trends of storage disaggregation, memory disaggregation is of increasing interest to the datacenter community [13, 15, 23, 28, 46]. Memory disaggregation offers the opportunity to move away from a server-centric approach for memory resources and instead provide a common allocatable set of memory resources to be used as needed. Disaggregation has been motivated by two driving application trends, addressing memory underutilization by reducing overprovisioning and leveraging shared memory to accelerate big data workloads that break the boundaries of a single server [13, 23] [22]. CXL has emerged as enabling technology for memory disaggregation, as the CXL.mem protocol

facilitates a multi-host, switched, design[15]. Pooled memory is emerging as a possible model to leverage memory disaggregation in the datacenter[28]. It consists of pooling memory resources from multiple servers over an interconnect network. With the support of shared FAM in CXL 3.X pooled memory permits multiple servers to transparently address the same memory region without any extra software layers. This enables applications to concurrently access the same memory regions, opening the door for the collaborative processing of working sets at scale all over networked access to shared memory.

2.3 Cache Coherency in CXL

The support for cache coherence has been a driving decision in the design of the CXL specification[39]. Since CXL 1.0, cache coherence has been a core feature, first present in the CXL.cache protocol. In CXL Type 1 devices this mechanism provides cache coherent support by extending a Host's coherence domain into the device memory. With Type 2 devices, CXL supports the CXL.mem protocol and with it Host managed memory with device managed coherence (HDM-DB). HDM-DB introduces the hardware cache coherence protocol for CXL devices; Back-Invalidate snoop coherence(BISnp). BISnp permits the device to track individual cache lines with an inclusive snoop filter. Thus permitting a cache coherence to be maintained in both a devices and host cache hierarchy. Not a sentence. ~~CXL Type 3 devices introduce Multi Logical Device's (MLD),~~ which allow the device to partition it's resources to function as 16 distinct logical devices (LD). MLD's enabled a CXL pooled memory architecture by permitting each LD to function as their own host endpoint for the underlying CXL device. MLD's support fabric attached memory by exposing host-managed device memory to multiple hosts over port based routing. With multiple host interfaces configured to access the device's memory concurrently host servers gain access to shared FAM in the same address space.

Shared FAM has two coherency models; multi-host hardware coherency or software-managed coherency. Multi-host hardware coherence extends the BISnp cache coherence protocol to support multiple concurrent hosts. With cross domain cache coherence, hosts are able to execute arbitrary atomic operations over an exclusively owned cache line. Atomic operations are critically important towards the efficiency of multi thread synchronization, yet their performance is constrained by the latency overheads of far memory and the overheads of managing cache coherence for multiple hosts. Not the right place to reveal this. A core goal of this work is the exploration of the cost of hardware cache coherence for synchronization at scale.

Software-managed coherence on the other hand avoids hardware supported cache coherence all together. Instead it relies on the application software layer to invalidate and/or flush cache hierarchies. How? Synchronization is still possible in this environment through the careful use of fences to enforce memory consistency and explicit memory operations. While it avoids the use of hardware cache coherency it limits inter-thread cross host synchronization to the use of traditionally less performant software solutions.

2.4 Mutual Exclusion

Mutual exclusion is a fundamental mechanism to control concurrent execution. It provides synchronization by enabling concurrent threads to time their accesses to shared resources. Their performance is highly reliant on the availability of synchronization primitives in the form of complex atomic instructions. This bifurcates mutual exclusion solutions into two distinct categories, software solutions which leverage assignment and control structures, and hardware solutions which leverage read-modify-write(rmw) hardware primitives. Also assignment

The performance of mutexes however is not only reliant on their underlying memory environment but also on the notification mechanism that each thread can use to "learn" if they have acquired a mutex. In modern multiprocessors the notification mechanism is often provided by the operating system kernel, which keeps track of lock state and enables sleep/awake patterns with threads acquiring the lock. Such a notification mechanism is fundamentally inaccessible in a CXL environment as there is no centralized monitor that can identify the lock state and wake up any waiting threads.

Thus we ~~are left~~ with the next best thing; polling. Polling involves each thread repeatedly monitoring a memory location for a value change. Both software and hardware mutexes rely on polling in this environment to "learn" of their place in their respective algorithms and whether they have made progress in the algorithm. This mechanism is key towards implementing mutexes in a far memory environment, yet it presents a notable performance hurdle as it saturates both CPU resources and the interconnect network with "redundant" read instructions. When applicable we implement exponential back-off techniques, in which a thread will increase the interval in which it polls a memory location in the case of successive misreads, to reduce the number of redundant polling. You could also invent some new notification mechanism...

Hardware and software locks must also contend with the reordering of load and store instructions by compiler and hardware optimizations. These optimizations seek to accelerate program execution by reordering loads and stores when possible, yet in the case of concurrent programs this can result in data races which violate mutual exclusion. Thus to ensure the correctness of our implementations we leverage fences(also called memory barriers) to ensure a consistent ordering of loads and stores. The inclusion of fences in a given algorithm can adversely impact performance, we implement each mutex solution with the minimal number of fences possible for each given implementation. This section is background!

Aside from the concern of performance, locks provide differing capabilities in terms of starvation freedom and fairness. For a lock to be starvation free a thread acquiring the

how does this look different? Code showing getting a mutex under different assumptions would help.

lock must eventually succeed. While fairness concerns itself with preventing waiting threads from being overtaken by threads that have arrived at a later point. Fairness explicitly requires the lock to establish an ordering among threads acquiring. If a lock is considered to be fair it is also starvation free. Where are the citations?

2.4.1 Software Locks.

The key characteristic of software locks is the minimal reliance on underlying hardware. This statement comes with a caveat however, as there is a distinction between software solutions that rely on atomic loads and stores and those that do not. As defined by Lamport [24], [25], we can classify these different solutions as RW-safe which do not rely on atomic read/write operations and RW-unsafe which do. While this distinction can seem entirely academic in the context of modern multiprocessor systems (as they provide atomic loads and stores, and support a cache coherency), it becomes relevant for far memory multi host CXL systems. In the current specification for software-managed coherence in CXL 3.X the support for atomic loads/stores is ambiguous. Therefore it is unclear whether such atomicity can be used to guarantee the correctness of a software mutex implementation. The lack of atomic loads and stores is problematic for multiport memory as simultaneous read/write operations can cause reads to flicker and writes to scramble the underlying memory. While such problems can be infrequent, the scale in which CXL memory systems are expected to grow exacerbate the risk significantly, as devices could support 16 ports for simultaneous reads/writes to the same memory address. ??

This is not a problem in the case of hardware coherence, as the cache coherence protocol can leverage the same technique it uses to provide read-modify-write atomicity to normal reads and writes. Support for atomic loads/store does not fundamentally require full cache coherent support, as there are multiple schemes that can resolve the conflicts of simultaneous reads/writes[43]. Whether CXL 3.X devices will support such schemes is unknown as of time of writing. Explorations of CXL for near memory compute can offer a solution to this problem by supporting conflict resolution schemes, yet this potential solution is speculative at this moment.

Previous work exploring synchronization of CXL memory[40]by leveraging software solutions cleverly mitigated this problem by implementing a modified Tournament Peterson lock. The authors leverage a binary tree of N-1 two thread Peterson locks to support N concurrent threads. This keeps the values of the concurrent shared variables that permit threads to proceed through the tournament constrained to zero or one, mitigating the risk of a flickered read or a scrambled write. In the light of this ambiguity around atomic support we implement a series of software solutions, some that are RW-Safe and some that are RW-Unsafe as outlined in Table. 1.

What is this, and why was it a good fit for that environment?

2.4.2 Hardware Locks.

In contrast to software locks, hardware locks require the underlying hardware to provide atomic primitives in order to establish mutual exclusion.

These primitives take the form of special atomic operations, which extend atomicity to three distinct operations; the initial memory read into a register, an optional sometimes data dependent action, and the final write back to the initial memory address. As no interruption of the operation can occur during this sequence of operations, these atomics are implemented as their own hardware instruction by a given CPU's ISA. For example, the test-and-set instruction will write a value to a memory location, and then return the old value for the caller to "test" whether the value had changed.

The power of these atomic instructions is made clear when considering the implementation of a shared counter. Without hardware atomics, one would need to guard a counter variable with a mutex to ensure that the increment occurs correctly, as without the atomicity guarantee multiple threads can read and attempt to increment the variable concurrently. With hardware atomics you can simply use the Fetch-and-add instruction, which will read a value from a memory address, increment it, and then write it back to its original address, all atomically.

Not all atomic instructions are made equal however, as outlined by Herlihy[17] there is a hierarchy of atomic objects in their relative capacity to solve the consensus problem for a certain number of processes. Similarly, the performance of atomic instructions is not equivalent to each other, and depends on its semantics and the given hardware implementation[38]. why are you telling us this?

This paper does not explore equivalent implementations of hardware locks with distinct hardware primitives, and makes no attempt to evaluate the relative performance between instructions. We justify this in two ways.

First, we decide on implementing hardware mutexes with the primitives they were originally proposed, as ensuring the correctness of new mutex implementations can be challenging. This is not background

Second, in a CXL hardware coherent environment the hardware primitive can be provided by distinct architectures, as the support for complex atomics relies on the hardware cache coherence protocol. We hypothesize that the primary cost in the performance of the hardware locks will be the underlying hardware cache coherence protocol with high latency, and not the relative performance of the underlying hardware primitives between them.

3 FPGA Hardware Cache Coherence Implementation with Programmable Delay Buffer

To prototype a Compute Express Link (CXL) Type-2 device on FPGA, we designed and implemented a hardware cache

Why? This section feels unconnected with everything that came before.

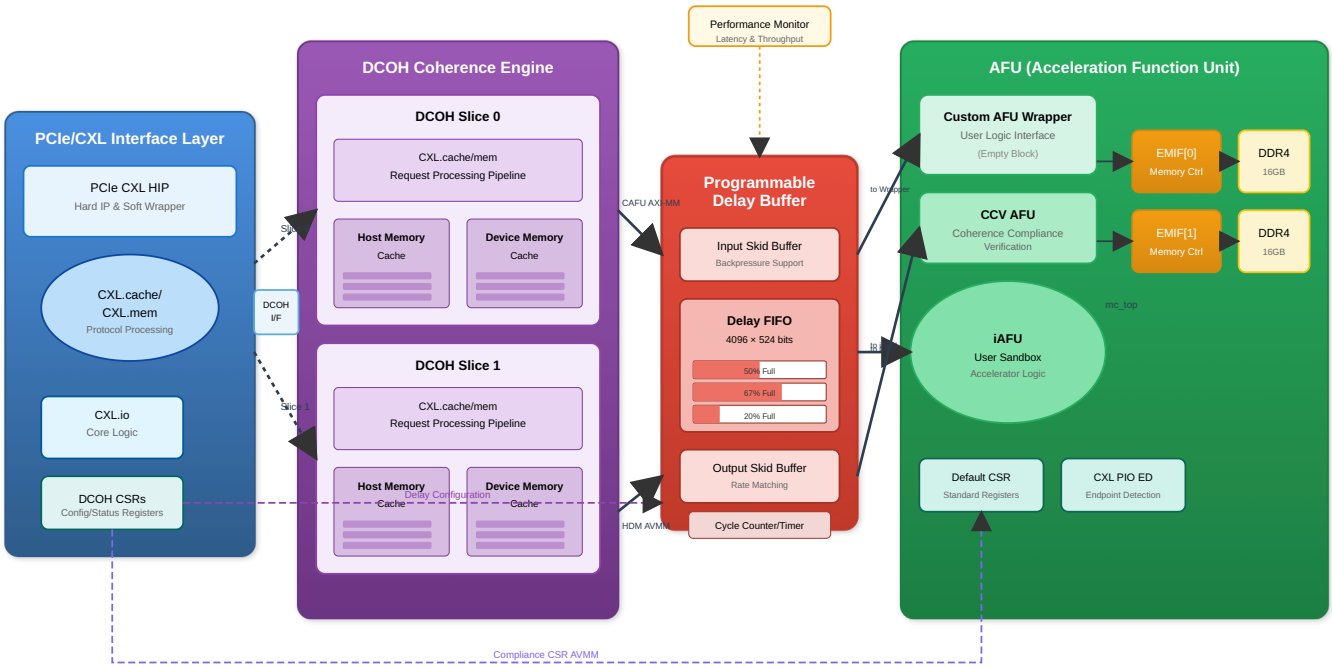


Figure 1. Delayed buffer Architecture

coherence engine with a ~~sophisticated~~ programmable delay buffer system that enables comprehensive evaluation of memory timing impacts on coherence protocols. The central innovation of our implementation is the integration of configurable delay injection mechanisms throughout the memory access path, allowing us to emulate various memory technologies and network conditions while maintaining full CXL protocol compliance. This approach builds upon existing CXL simulation frameworks such as CXLMemSim[45], extending them with hardware-validated timing-accurate coherence mechanisms. Why was it necessary to build this?

3.1 Programmable Delay Buffer Architecture

The core of our implementation centers on a programmable delay buffer that intercepts and controls the timing of memory responses in the CXL coherence path. As shown in the provided source code, this delay buffer is implemented as a ~~sophisticated~~ FIFO-based queuing system that timestamps incoming read responses and releases them after a configurable delay period. The implementation, written in Chisel and compiled to SystemVerilog, consists of three primary components: an input skid buffer for flow control, a deep delay FIFO for timing management, and an output skid buffer for interface decoupling.

The delay buffer operates on 524-bit wide data paths, accommodating the full 512-bit data payload plus 12 bits of metadata including transaction ID, response status, and last-beat indicators. Each entry in the delay FIFO is augmented with a 12-bit timestamp field that records the target release cycle for that particular response. The timestamp calculation occurs at enqueue time, adding the current cycle counter value to the programmed delay value from the configuration register. This approach ensures consistent delay application regardless of queue occupancy or system load conditions.

The implementation employs a 4096-entry deep FIFO structure, utilizing FPGA BRAM resources efficiently through the use of synchronous read memories with output pipelining. This depth allows the buffer to accommodate up to 4096 outstanding transactions, sufficient for maintaining full bandwidth utilization even with delays extending to thousands of cycles. The FIFO implementation leverages Chisel's built-in Queue primitive with specific parameters for synchronous memory inference and pipelined reads, ensuring optimal mapping to FPGA memory resources while maintaining single-cycle throughput for enqueue and dequeue operations.

What does this mean

3.2 Skid Buffer Implementation for Timing Isolation

The skid buffers at the input and output interfaces serve a critical role in maintaining timing closure and providing elastic buffering for flow control. ~~Our skid buffer~~ implementation follows a two-register design pattern that allows for full throughput while accommodating back-pressure scenarios. The input skid buffer accepts incoming AXI read responses and provides them to the delay FIFO, managing the ready-valid handshake protocol to ensure no data loss even when the delay FIFO temporarily cannot accept new entries.

The skid buffer maintains three key state elements: a ready register that indicates whether the buffer can accept new data, a valid register tracking whether buffered data needs to be transmitted, and a data register holding the buffered transaction. The control logic implements a sophisticated state machine that handles all possible combinations of input arrival and output consumption, ensuring that data can flow through with minimal latency when no back-pressure exists while still maintaining full protocol compliance during congestion events. This design achieves the critical property of combinational passthrough when the output is ready, minimizing latency in the common case while providing single-cycle buffering when needed.

The output skid buffer performs a similar function but with additional complexity due to the delay release mechanism. It monitors the head of the delay FIFO and only considers entries valid when their timestamp matches or precedes the current cycle counter. This temporal gating ensures that responses are held for exactly the programmed delay period, neither more nor less, maintaining precise timing control essential for our coherence protocol experiments.

3.3 Delay Control and Configuration Infrastructure

The delay value is dynamically configurable through a memory-mapped control register accessible via the CXL.io configuration space. The implementation places this control register at offset 0xE08 within the extended capability structure, making it discoverable through standard PCIe configuration space walks. The 32-bit delay register allows specification of delays ranging from 0 to over 4 billion cycles, though practical values typically range from 32 to 2048 cycles corresponding to memory latencies from local DRAM to remote NUMA nodes.

The configuration infrastructure includes not only the delay value register but also associated status and control registers that provide visibility into buffer operation. A status register at offset 0xE04 reports the current buffer occupancy, maximum observed occupancy, and overflow indicators. This information proves invaluable during experimental runs, allowing us to verify that the buffer depth is sufficient for the configured delay and workload characteristics. The control register at offset 0xE00 includes enable bits for delay injection, options for selective delay application based on transaction type, and reset controls for the statistics counters.

The extended capability header at offset 0xE00 follows the PCIe extended capability format with a capability ID of 0x23 (vendor-specific) and includes proper version and next-pointer fields for integration into the device's capability chain. This standard-compliant implementation ensures that the delay injection mechanism is discoverable and configurable using standard PCIe configuration tools, facilitating integration into existing test frameworks and experimental setups.

3.4 Timing Management and Cycle Counter Design

The cycle counter that drives the delay buffer timing represents a critical component requiring careful design consideration. Implemented as a 12-bit free-running counter, it provides sufficient range to support delays up to 4096 cycles while maintaining reasonable resource utilization. The counter operates in the same clock domain as the AXI interfaces (400 MHz in our implementation), ensuring consistent timing relationships between enqueue timestamps and dequeue decisions.

Counter overflow handling required particular attention to ensure correct operation when the counter wraps. The comparison logic for timestamp checking implements modular arithmetic, correctly handling cases where the target timestamp is numerically less than the current counter value due to wraparound. This implementation ensures that delays remain consistent even during long-running experiments where the counter may wrap multiple times.

The timing management system also includes provisions for clock domain crossing between the delay buffer operating at 400 MHz and the CXL interface running at higher speeds. Asynchronous FIFOs with Gray code pointers handle this transition, ensuring reliable data transfer while maintaining the timing relationships established by the delay buffer. The clock domain crossing logic includes additional buffering to accommodate rate mismatches, preventing underflow or overflow conditions that could corrupt the delay timing.

3.5 Integration with CXL Memory Subsystem

The delay buffer integrates seamlessly into the broader CXL memory subsystem through careful interface design and signal routing. In the AFU (Accelerator Functional Unit) top-level module, the delay buffer instantiation occurs within a generate loop that creates separate delay buffer instances for each memory channel. This per-channel architecture ensures that delay injection can be applied uniformly or differentially across channels, enabling experiments with non-uniform memory architectures.

The integration maintains full AXI4 protocol compliance while inserting the delay buffer into the read response path. Write operations bypass the delay buffer entirely, flowing directly from the CXL interface to the memory controllers. This asymmetric treatment reflects the reality that read latency typically dominates performance in memory-intensive workloads, while write operations can often be posted and overlapped with computation. The bypass path for writes includes its own flow control logic to ensure that write and read operations maintain proper ordering relationships despite following different paths through the system.

Signal routing between the delay buffer and surrounding logic employs careful bit-field extraction and concatenation to maintain the full AXI response information. The 524-bit internal data path decomposes into the 512-bit read data,

8-bit transaction ID, 1-bit last indicator, 2-bit response status, and 1-bit user field. These fields are properly extracted at the delay buffer output and routed to the appropriate AXI response channel signals, ensuring that all protocol information is preserved through the delay injection process.

3.6 Debug and Observability Features

The implementation includes comprehensive debug capabilities essential for understanding delay buffer behavior and diagnosing issues. When synthesis is performed for hardware rather than simulation (controlled by the VCSSIM define), the design instantiates a ChipScope debug core that provides real-time visibility into buffer operations. The debug interface monitors 256 bits of carefully selected signals including input and output transaction details, buffer occupancy, timestamp comparisons, and flow control status.

The debug signal selection focuses on three primary areas of interest. First, the input interface signals including address, valid, ready, and transaction ID allow observation of incoming traffic patterns and back-pressure events. Second, the internal buffer state signals including occupancy count, head and tail pointers, and timestamp comparison results provide insight into buffer dynamics and delay accuracy. Third, the output interface signals confirm proper delay application and response delivery. This comprehensive debug visibility proved essential during development for identifying and resolving subtle timing issues related to timestamp comparison and overflow handling.

Performance counters integrated into the delay buffer track key metrics including total transactions processed, average buffer occupancy, maximum observed latency, and frequency of back-pressure events. These counters are accessible through memory-mapped registers and can be sampled periodically during experiments to understand system behavior under different delay configurations and workload patterns.

3.7 Experimental Applications and Use Cases

The programmable delay buffer enables several critical experimental capabilities for our CXL coherence research. For evaluating synchronization primitives, we can inject controlled delays to stress-test lock implementations and identify timing-dependent correctness issues. The ability to vary delay dynamically during execution allows us to evaluate adaptive algorithms that adjust their behavior based on observed memory latency.

In studying coherence protocol scalability, the delay buffer allows us to emulate systems with varying memory hierarchies and interconnect topologies. By configuring different delays for different address ranges or transaction types, we can model complex NUMA architectures and evaluate how coherence protocols perform as memory latency increases. This capability proved particularly valuable for validating our software-managed coherence mechanisms, which must remain correct despite wide variations in memory response timing.

The delay injection mechanism also facilitates performance debugging by allowing us to identify latency-sensitive portions of applications. By gradually increasing delay and observing performance degradation, we can pinpoint code sections that would benefit from prefetching, caching, or algorithmic improvements. This technique has revealed several unexpected performance bottlenecks in our test applications that were not apparent under uniform low-latency conditions.

3.8 Synthesis Results and Resource Utilization

The complete delay buffer implementation, including all skid buffers, the delay FIFO, control logic, and debug infrastructure, consumes approximately 8% of the FPGA LUT resources and 15% of the BRAM resources on our Intel Stratix 10 target device. The design achieves timing closure at 400 MHz with all paths meeting setup and hold requirements with positive slack. The critical path runs through the timestamp comparison logic, which we optimized using parallel comparison trees and pipeline registers.

Resource utilization scales linearly with the number of memory channels, as each channel requires its own delay buffer instance. For our eight-channel configuration, the total delay buffer infrastructure consumes approximately 35% of BRAM resources, primarily due to the deep FIFO requirements. This resource usage represents an acceptable trade-off for the experimental capabilities provided, though production implementations might use smaller buffers or share buffers across channels to reduce resource consumption.

The synthesis process required several optimization iterations to achieve efficient BRAM inference for the delay FIFO. Initial implementations using behavioral memory descriptions resulted in inefficient LUT-based storage. By explicitly using Chisel's synchronous memory primitives and providing appropriate synthesis attributes, we achieved optimal BRAM mapping with single-cycle read latency and full bandwidth utilization.

4 Methodology

of what? So far, we had background and the description of an emulator.

We evaluate the synchronization mechanisms using a microbenchmark that measures lock acquisition and release latency under varying contention levels. Each benchmark consists of multiple threads competing for lock acquisition, performing a critical section operation, and then releasing the lock.

The benchmark parameters include:

- Thread count: 1 to 64 threads to evaluate scalability
- Critical section duration: Minimal (empty) to isolate synchronization overhead
- Measurement duration: 20 seconds per configuration with 1-second warmup

Your terminology? or standard def'ns?

Lock Name	Coherence	Starve-Free	Fair	RW-Safe
Bakery Algorithm[26]	Software	Yes	Yes	Yes
Boulangerie[1]	Software	Yes	Yes	Yes
Lamport Fast[27]	Software	No	No	No
Elevator-LF-Array[6]	Software	Yes	Yes	No
Elevator-BL-Array[6]	Software	Yes	Yes	Yes
Elevator-LF-Tree[6]	Software	Yes	Yes	No
Elevator-BL-Tree[6]	Software	Yes	Yes	Yes
Peterson[35]	Software	No	Yes	No
Knuth[21]	Software	No	Yes	No
Spin[3]	Hardware	No	No	No
Exp Spin[3]	Hardware	No	No	No
Ticket[32]	Hardware	Yes	Yes	No
MCS[32]	Hardware	Yes	Yes	No

Table 1. Lock Characteristics. Software coherence locks rely solely on memory ordering guarantees without hardware cache coherence, while hardware coherence locks leverage cache coherence protocols for synchronization.

- Memory placement: Locks allocated in both local DRAM and CXL-attached memory

We measure throughput (successful lock acquisitions per second) and fairness (standard deviation of per-thread acquisition counts) for each configuration.

hm

We evaluate the mutex implementation by leveraging two benchmarks, a max contention benchmark and min contention benchmark. Lock contention by multiple threads significantly affects their performance in terms of time to acquire a lock. The longer the delay for a thread to acquire a lock is, the worse it can be considered to perform as it prevents the thread from accessing the critical section and prevents it from carrying out other useful work. As all our implementations rely on polling to acquire the lock this delay takes the form of an unbounded number of memory operations, as locks will continue to poll until they succeed. We select a max contention and a min contention benchmark to explore the two extreme ends of lock contention, thus providing an upper and lower bound of performance for each implementation.

The max contention benchmark creates N test threads that wait on a shared flag. The test harness then flips the flag, and the N threads concurrently attempt to acquire the lock for a fixed window of configurable time t. When a thread succeeds on acquiring the lock they increment a shared counter in the critical sections and a per thread individual counter, after which they release the lock. We utilize the shared counter to indicate the total number of successful acquisitions of the lock in a fixed window of time. The higher the number of lock acquisitions the better, as a lock that can support more lock acquisitions is less vulnerable to the performance degradation of increasing contention. This approach permits us to evaluate the worst case scenario of lock contention with minimal impact of instrumentation to our experiment. We parametrize this benchmark over an increasing number of threads to get an idea of how a lock evolves with increasing contention.

In order to mitigate the impact that a thread_id might have on an individual thread's access time(due to certain implementations bias towards lower thread_ids when tie braking), we randomize thread ID's (in the range of 1 - N) at startup for the executing threads between experiments.

For the Max Benchmark for every lock at a given level of concurrency we run each experiment 5 times for 20 seconds. For the Min Benchmark we instead run the experiment for a 1 second duration, this is due to the single threaded performance not being as vulnerable to performance scheduling effects.

4.1 Mutex Implementations

We select the following mutexes in order to explore a wide selection of different behaviors that could impact performance. We outline their characteristics in Table. 1.

In terms of the software locks we test the Bakery Algorithm, the Lamport-Fast Algorithm, the Burns-Lamport Algorithm, and the Elevator Algorithm series as proposed by Buhr. et Al. The Bakery Algorithm establishes a baseline of software lock performance while providing RW-Safety and thus not requiring atomic loads and stores. We include a variant of the Bakery Algorithm known as the Boulangerie lock[1] which avoids certain unnecessary read operations. We include the Peterson [35] and Knuth[21] as classical software locks. The Lamport-Fast[27] algorithm utilizes the smallest number of of load/store operations in the uncontested case but is RW-Unsafe. The Burns-Lamport(A modification of the Burns-Lynch[7] lock by Lamport and verified by Hesselink [18]) algorithm provides RW-Safety, and serves as a mutex primitive for half of the testes elevator algorithms. We test

?

the Elevator Algorithms as designed by Buhr. et Al[6], as they are modern implementations of software locks that leverage a contention avoiding design. They function by having a thread wake up it's successor in the unlock procedure by either doing a linear array search or a tree search accordingly. They leverage an underlying Lamport-Fast lock or Burns-Lamport lock to provide fundamental mutual exclusion, while leveraging the elevator procedure to reduce contention on said lock.

In terms of hardware locks we test two Spin Locks, a Ticket Lock, and the MCS lock[32]. The spin locks and ticket locks are of interest due to their reliance on a single shared variable to ensure mutual exclusion supported by rmw atomics. The spin locks leverage the test-and-set atomic operation to ensure concurrent access, continuously polling creates significant traffic to a shared cache line and saturates the processor with operations. The exp_spin lock exhibits improved performance by reducing the overall cache line contention by engaging in exponential back off with recurring acquire "misses" This overhead is reduced in standard multiprocessor spin lock implementations by leveraging the kernel to deschedule a thread after it has failed in acquiring a lock and waking it up once it can reattempt. Without a centralized manager in a far memory environment that can monitor thread behavior and trigger sleep/awake patterns in contending threads such a technique is unfeasible to replicate. The ticket lock has a similar access pattern but instead leverages a fetch-and-add atomic to grant a ticket to a waiting thread(which is then polled on by it's owner), and then leverages a shared fetch-and-add in the unlock path to grant a ticket to the next thread.

The MCS lock on the other hand avoids heavily contending on a single cache line by queuing per thread lock objects in the lock path that then are polled on exclusively by it's corresponding thread owner. They proceed by being notified by an unlocking thread that they can acquire the lock. This drastically reduces shared cache line contention notably improving performance. The MCS lock leverages the compare-and-swap operation to ensure the "tail" of the queue of lock objects is safe when threads attempt to concurrently queue themselves.

5 Evaluation

5.1 Experimental Setup

Our experimental platform is built on the BittWare IA-780i accelerator card, which integrates Intel's CXL IP core to provide hardware-accelerated CXL functionality. The FPGA features an Intel Agilex 7 FPGA, offering the computational flexibility needed to implement and evaluate different coherence models specified in the CXL 2.0 standard. The card connects to the host system through a PCIe 5.0 x16 interface, providing the high bandwidth required for memory-intensive workloads while maintaining compatibility with CXL's physical layer requirements. The FPGA is configured to operate as a CXL Type 2 device, exposing 32GB of onboard DDR4 memory as a CXL-attached memory pool to the host system. This configuration allows us to evaluate both software-managed and hardware-managed coherence protocols in a realistic CXL memory expansion scenario. The Intel CXL IP core handles the protocol layer operations, including memory transactions, coherence management, and the implementation of CXL.mem and CXL.cache protocols necessary for our synchronization primitives.

To comprehensively evaluate the performance characteristics of different synchronization mechanisms under varying network conditions, we implemented a configurable delay buffer within the FPGA fabric. This delay buffer sits in the data path between the CXL controller and the memory controller, allowing us to inject programmable latencies that simulate different CXL deployment scenarios. The delay buffer supports a programmable range from 50 nanoseconds to 10 microseconds with a granularity of 10 nanoseconds, enabling fine-grained control over access latencies. Furthermore, we can independently configure read and write delays, which is crucial for understanding how different synchronization algorithms respond to asymmetric memory access patterns common in distributed memory systems.

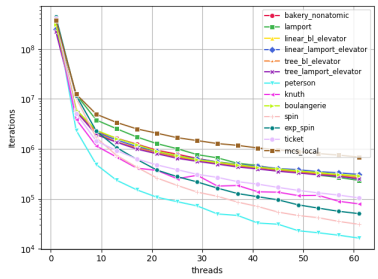
The host system consists of a Xeon 6 Granite Rapids 6787P, paired with 256GB of DDR5 MRDIMM memory. The system runs Linux kernel version 6.x with CXL driver support enabled, providing the necessary software stack for CXL device enumeration and memory management. This setup allows us to treat the FPGA-attached memory as part of the system's memory hierarchy, accessible through standard load/store operations for software-managed coherence or through atomic operations for hardware-managed coherence.

For our experiments, we evaluate 3 distinct latency configurations to represent different CXL deployment scenarios. The DRAM configuration provides a baseline level of performance. The low-latency configuration of 400 nanoseconds simulates locally attached CXL memory within the same chassis. We test an extreme-high latency scenario of 10 microseconds to understand the behavior of synchronization primitives under worst-case network conditions. These varied configurations allow us to systematically characterize how software and hardware synchronization mechanisms scale across different CXL fabric topologies and distances.

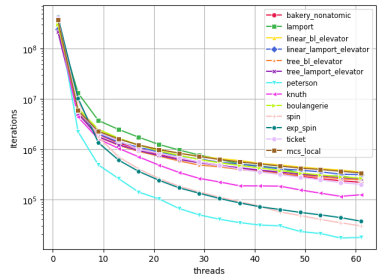
5.2 Results

Our results evaluate the total number of iterations achieved concurrently by participating threads for a given benchmark. We increment from 1 thread to 64 in jumps of 4 threads.

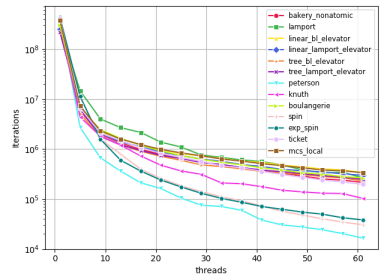
5.2.1 Max Benchmark. The Max Benchmark is represented by figures 2a, 2b, and 2c.



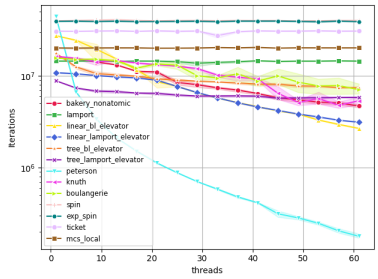
(a) Max Bench: DRAM Baseline



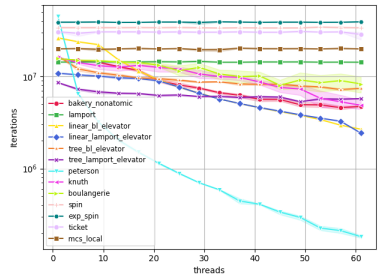
(b) Max Bench: CXL 400ns



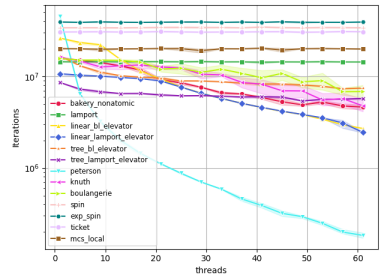
(c) Max Bench: CXL 10us



(d) Min Bench:DRAM Baseline



(e) Min Bench: CXL 400ns



(f) Min Bench: CXL 10us

Where is the caption? What am I looking at?

The max benchmark aims to maximally pressure the lock implementations by having all threads at a given level of concurrency attempt to acquire the lock.

Inspecting the baseline performance of the locks in a DRAM environment we observe consistent behavior in the "naive"(spin, exp_spin, ticket) hardware locks that is maintained across each latency benchmark.

The overhead of polling is notable, and demonstrates itself as significant overhead in our DRAM baseline in Figure.2a in the low performance of the hardware locks exp_spin, and spin. This overhead is incurred by the ticket lock as well, exhibiting similar performance characteristics to the spin locks.

The MCS lock notably provides the best lock performance in DRAM. It's contention avoidant design permits it to reduce the resulting coherence traffic that adversely affects the spin locks and ticket lock. The MCS lock conclusively outperforms every software lock in our baseline.

Among the software locks we observe Peterson [35] and Knuth[21] perform the worst in the set. Both locks heavily rely on scanning the status of other threads, and on multiple reattempts per thread in order to acquire the lock.

The Bakery[26] and Boulangerie[1] locks carry out similar scanning patterns but threads do not retry once they've begun to acquire the lock as they algorithm provide them a fairness guarantee that establishes an access ordering.

The Lamport Fast[27] lock performs surprisingly well until it reaches the upper ends of concurrency.

Notably the software locks closest to MCS in terms of performance are the elevator series described by Buhr. et Al [6]. This close performance to MCS is similar as found in the original work that proposed their design.

The observed dynamics change once we allocate on CXL memory. In these deployments hardware coherence is provided to cache lines with multiple writers. As most software locks maintain a single writer principle for many of their shared variables that report on thread state these variables are supported by software coherence exclusively. This is not the case with the hardware locks whose algorithms entirely rely on requiring hardware cache coherence to support any rmw atomic operation they need.

As seen on Figures 2b and 2c we observe a changing performance of hardware locks.

The spin locks maintain their relative position, yet interestingly the exp_spin lock provides minimal performance benefit. The hardware coherence overhead is likely responsible for this convergence as both locks are constrained by its underlying performance.

Interestingly, the ticket lock performs a notably better, becoming competitive with the software locks. As the primary polling traffic of the ticket lock only requires a simple load operation and not a rmw atomic, this likely reduces the overall coherence traffic to the fetch-and-add operations that increment the ticket counter. This pattern might actually be benefiting from the hardware coherence traffic as once a thread acquires the lock, it'll be the only writer to the shared counter among all the polling threads.

Couldn't ya confirm this?

Notable the MCS lock is no longer the best performing lock, with the Linear-BurnsLynch-Elevator lock outperforming it at high contention in both 2b and 2c.

While the Linear-Lamport-Elevator relies on a pair of shared read/write variables to guarantee mutual exclusion (as it leverages the Lamport-Fast Algorithm as it's underlying mutex) which would be subjected to hardware coherence the Linear-BurnsLynch-Elevator does not. As the latency is increased from Figure 2b to 2c we do not observe significant changes in the performance between the locks.

5.3 Min Benchmark

The Min Benchmark seeks to evaluate the locks in the ideal scenario, only a single thread acquiring the lock at a time. This establishes a lower performance bound, and allows us to observe the inherent overheads of extending a lock to support multiple threads.

Naturally these overheads are only present in the software locks, as the hardware locks do not require more variables to be allocated to support more threads.

This is observable in Figure 2d, with the hardware locks mainting stable high performance throughout, independent of the number of threads that end up accessing them. This is not the case with software locks which exhibit degrading performance due to requiring each thread to scan more variables to account for the extra allocated variables for all the "participating" threads. In this space the best performing software lock being the Lamport-Fast algorithm. This is correlated with it's intended design, which sought to reduce the lock's number of load and stores to the theoretical minimum in the uncontested case.

The overall performance of both software and hardware locks do not notably change when allocated in CXL memory as seen in Figures 2e and 2f. As the Min benchmark only exercises a single thread accessing the lock at a time, there is minimal cache line contention. The only scenario in which cache line contention occurs, and the corresponding hardware coherence overhead is when the acquiring thread switches. As there only two threads when this occurs the overhead is minimal. Interestingly the most notable difference between our baseline and CXL allocation is the improved performance of exp_spin over the traditional spin lock.

5.4 Discussion

These results provide an initial exploration of the performance of synchronization mechanisms in software-managed coherence and hardware coherence in CXL memory.

They indicate that the performance of the hardware coherence protocol can create overhead for synchronization mechanisms in CXL FAM environments, and that software managed synchronization offers competitive performance.

While it's clear that hardware coherence provides key mechanisms for synchronization, and enables performant solutions at certain levels of traffic, it is not the only viable mechanism to achieve efficient synchronization.

This is particularly useful due to the potential limits of supporting hardware coherence with increasing memory This is particularly useful due to the potential limits of supporting hardware coherence with increasing memory scale, as maintaining cache coherence across large memory pools introduces significant complexity and overhead [15]. Providing a mechanism to achieve synchronization without pressuring the limited support for hardware coherence will be key for large scale applications that seek to support collaborative workloads over FAM.

A key takeway is that independent of the chosen synchronization mechanism it is key to reduce cache line contention in a far memory environment. The most efficient software and hardware solutions found leveraged distinct designs to provide that spread out thread memory accesses in such a way that cacheline contention was rare and only occured when necessary.

A curious result of our experiments is the limited impact that high read latency has on the performance of our solutions. This can be attributed to the predominant use of single-writer patterns in many lock implementations, where shared variables are modified by only one thread at a time. In such designs, the critical path often involves write operations and cache invalidations rather than read operations. Additionally, modern processors' ability to hide read latency through out-of-order execution and prefetching may further mitigate the impact of increased read latency in CXL memory environments.

6 Related Work

There is one work that has explored software synchronization in CXL memory[40]. They explore software locks in comparison to the execution of a single rmw atomic. While this provides insight into the overhead that carrying out synchronization in far memory involves, it does not provide a view of the potential overheads that hardware coherence can introduce to provide access to a rmw atomic. Their clever use of a Peterson tournament lock highlights the need for synchronization mechanisms to spread contention across the whole data structure, a result that our results reproduce.

Other work has discussed the need for synchronization mechanisms that do not rely on hardware coherence in a CXL environment [20] [2]

7 Future Work

This work only provides an initial explorations of the complex tradeoffs between coherence mechanisms in FAM for CXL memory.

A thorough exploration of the impact of distinct lock implementations with real applications will be instrumental towards highlighting the real performance tradeoffs between software-managed coherence and hardware coherence. This exploration must take into account not only low level shared

data structures such as caches and indexes, but also full applications such as databases and distributed batch processing. Recent work has highlighted that the best performing lock is highly dependent on the number of threads and the particular characteristics of a given application [16]. We expect that this will continue to be the case in a CXL FAM environment.

There's been significant progress over the last two decades exploring lock implementations for NUMA systems[8, 10, 29, 44]. A NUMA environment shares many similarities with a CXL environment, lessons learned in this domain are likely to be transferrable to the development of novel CXL specific lock implementations. A key obstacle in retrofitting NUMA locks to CXL will be defining similar abstractions around memory locality that NUMA locks leverage to improve performance. Many NUMA locks leverage a series of "cohorting" techniques[11], in which threads in a local NUMA node will contend with a centralized lock as a cohort, reducing cross node traffic and once a member thread succeeds in acquiring the lock enabling a fast path for the rest of the threads in the cohort. Such a technique requires requires retrofitting to a switchable CXL environment, in which data locality will depend on the specific topology of the CXL network and not to an explicit node.

A key driver in the performance of the mechanisms explored is the clear overhead of polling. Exploring lock implementations that utilize different notification mechanisms will further illuminate the trade offs between different locks. There have been limited proposals on how to achieve efficient notification schemes in CXL such as leveraging PCiE device interrupts, mwait polling instructions[30], and hybridizing network for sleep/awake patterns with in memory polling[20], but no actual implementations as of time of writing. This remains a key problem in providing scalability for CXL shared FAM architectures, as polling overhead threaten to undermine the significant benefits in shared memory CXL workloads [31].

Of significant interest is the performance of lockfree and waitfree data structures in CXL memory environments [17]. These data structures leverage rmw atomics to implement sophisticated concurrency mechanisms that are impossible to achieve with simple load/store instructions. Exploring implementations of producer/consumer queues with equivalent semantics but with contrasting implementations leveraging a lock-free design supported by rmw atomics or leveraging software locks for synchronization would provide insight into the distinct concurrency advantages that rmw semantics provides.

8 Conclusion

This work presents the first comprehensive evaluation of mutex implementations across CXL 3.0's two distinct coherence models: software-managed coherence using load/store operations with fences, and hardware-managed coherence leveraging read-modify-write atomics with back-invalidation snooping. Through implementation and testing on a BittWare IA-780i FPGA platform with Intel CXL IP, including the first FPGA-based implementation of CXL's hardware coherence protocol with back-invalidation support, we identify critical performance trade-offs.

Software locks demonstrate predictable scaling characteristics independent of cache coherence traffic but suffer from O(N log N) memory overhead and higher base latency. In contrast, hardware locks achieve superior performance at low latencies through cache-line efficiency but degrade significantly as fabric latency increases. We identify specific crossover points where software-managed coherence becomes competitive with hardware coherence. This suggests that applications can strategically avoid the complexity of full hardware coherence for certain synchronization patterns, particularly in high-latency CXL fabric deployments.

These findings provide essential guidance for: System designers choosing between CXL coherence models based on expected deployment topologies. Developers optimizing synchronization primitives for disaggregated memory architectures. Understanding when the overhead of hardware cache coherence may outweigh its benefits. The research demonstrates that while hardware coherence enables powerful synchronization primitives, it is not always the optimal choice in CXL FAM environments. The performance characteristics of both approaches must be carefully considered based on fabric latency, contention levels, and application requirements. This initial exploration establishes a foundation for further research into synchronization mechanisms optimized specifically for CXL's unique memory disaggregation capabilities.

References

[1] *Under the Hood of the Bakery Algorithm: Mutual Exclusion as a Matter of Priority*, pages 399–413. Springer International Publishing, Cham, 2015.

[2] Peter Alvaro, Matthew Adiletta, David Cheng, Adrian Cockroft, Frank Hady, Ramesh Illikkal, Esteban Ramos, and Robert Soulé. Notnets: Accelerating microservices by bypassing the network. In *Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems*, APSys '24, page 67–73, New York, NY, USA, 2024. Association for Computing Machinery.

[3] T.E. Anderson. The performance of spin lock alternatives for shared-memory multiprocessors. *IEEE Transactions on Parallel and Distributed Systems*, 1(1):6–16, January 1990.

[4] Pavan Balaji, Darius Buntinas, David Goodell, William Gropp, and Rajeev Thakur. Toward Efficient Support for Multithreaded MPI Communication. In Alexey Lastovetsky, Tahar Kechadi, and Jack Dongarra, editors, *Recent Advances in Parallel Virtual Machine and Message Passing Interface*, volume 5205, pages 120–129. Springer Berlin Heidelberg, Berlin, Heidelberg, 2008.

[5] Silas Boyd-Wickizer, M Frans Kaashoek, Robert Morris, and Nickolai Zeldovich. Non-scalable locks are dangerous.

[6] Peter A. Buhr, David Dice, and Wim H. Hesselink. High-contention mutual exclusion by elevator algorithms. *Concurrency and Computation: Practice and Experience*, 30(18):e4475, 2018.

[7] JE Burns and NA Lynch. Allertonconf. In *Proceedings of the 18th Annual Allerton Conference on Communications, Control and Computing, Monticello, Illinois, USA, 1980; 833–842.*

[8] Milind Chabbi, Michael Fagan, and John Mellor-Crummey. High performance locks for multi-level NUMA systems. In *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP 2015, pages 215–226, New York, NY, USA, January 2015. Association for Computing Machinery.

[9] Hoang-Vu Dang, Sangmin Seo, Abdelhalim Amer, and Pavan Balaji. Advanced Thread Synchronization for Multithreaded MPI Implementations. In *2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, pages 314–324, Madrid, Spain, May 2017. IEEE.

[10] Dave Dice and Alex Kogan. Compact NUMA-Aware Locks, March 2019.

[11] David Dice, Virendra J Marathe, and Nir Shavit. Lock Cohorting: A General Technique for Designing NUMA Locks.

[12] Gábor Dózsa, Sameer Kumar, Pavan Balaji, Darius Buntinas, David Goodell, William Gropp, Joe Ratterman, and Rajeev Thakur. Enabling Concurrent Multithreaded MPI Communication on Multicore Petascale Systems. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Doug Tygar, Moshe Y. Vardi, Gerhard Weikum, Rainer Keller, Edgar Gabriel, Michael Resch, and Jack Dongarra, editors, *Recent Advances in the Message Passing Interface*, volume 6305, pages 11–20. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010.

[13] Mohammad Ewais and Paul Chow. Disaggregated Memory in the Datacenter: A Survey. *IEEE Access*, 11:20688–20712, 2023.

[14] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 57–70, Sydney NSW Australia, August 2024. ACM.

[15] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. Direct Access, High-Performance Memory Disaggregation with DirectCXL.

[16] Hugo Guiroux and Renaud Lachaize. Multicore Locks: The Case Is Not Closed Yet.

[17] Maurice Herlihy. Wait-free synchronization. *ACM Trans. Program. Lang. Syst.*, 13(1):124–149, January 1991.

[18] Wim H. Hesselink. Verifying a simplification of mutual exclusion by Lycklama–Hadzilacos. *Acta Informatica*, 50(3):199–228, May 2013.

[19] Torsten Hoefler, Duncan Roweth, Keith Underwood, Bob Alverson, Mark Griswold, Vahid Tabatabaee, Mohan Kalkunte, Surendra Anubolu, Siyuan Shen, Abdul Kabbani, Moray McLaren, and Steve Scott. Datacenter Ethernet and RDMA: Issues at Hyperscale, April 2023.

[20] Jaewan Hong, Marcos K. Aguilera, Emmanuel Amaro, Vincent Liu, Aurojit Panda, and Ion Stoica. The Dawn of Disaggregation and the Coherence Conundrum: A Call for Federated Coherence, April 2025.

[21] Donald E. Knuth. Additional comments on a problem in concurrent programming control. *Commun. ACM*, 9(5):321–322, May 1966.

[22] Dario Korolija. Farview: Disaggregated Memory with Operator Offloading for Database Engines.

[23] Youngeun Kwon and Minsoo Rhu. A Disaggregated Memory System for Deep Learning. *IEEE Micro*, 39(5):82–90, September 2019.

[24] L Lamport. The Mutual Exclusion Problem Part I: A Theory of Interprocess Communication.

[25] L Lamport. The Mutual Exclusion Problem Part II: Statement and Solutions.

[26] Leslie Lamport. A New Solution of Dijkstra's Concurrent Programming Problem. 17(8), 1974.

[27] Leslie Lamport. A fast mutual exclusion algorithm. *ACM Trans. Comput. Syst.*, 5(1):1–11, January 1987.

[28] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 574–587, Vancouver BC Canada, January 2023. ACM.

[29] Victor Luchangco, Dan Nussbaum, and Nir Shavit. A Hierarchical CLH Queue Lock. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Dough Tygar, Moshe Y. Vardi, Gerhard Weikum, Wolfgang E. Nagel, Wolfgang V. Walter, and Wolfgang Lehner, editors, *Euro-Par 2006 Parallel Processing*, volume 4128, pages 801–810. Springer Berlin Heidelberg, Berlin, Heidelberg, 2006.

[30] Teng Ma, Chengkun Wei, Haoyu Li, and Mingxing Zhang. HydraRPC: RPC in the CXL Era.

[31] Suyash Mahar, Ehsan Hajyjasini, Seungjin Lee, Zifeng Zhang, Mingyao Shen, and Steven Swanson. Telepathic Datacenters: Fast RPCs using Shared CXL Memory, August 2024.

[32] John M. Mellor-Crummey and Michael L. Scott. Algorithms for scalable synchronization on shared-memory multiprocessors. *ACM Trans. Comput. Syst.*, 9(1):21–65, February 1991.

[33] Haojun Zhao Phuc Nguyen Mohamed Mekkouri Leandro Werra Thomas Wolf Nouamane Tazi, Ferdinand Mom. The ultra-scale playbook: Training llms on gpu clusters, 2025.

[34] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting, May 2024.

[35] G. L. Peterson. Myths about the mutual exclusion problem. *Information Processing Letters*, 12(3):115–116, June 1981.

[36] Esteban Ramos. EIRNf/mutex_benchmark, September 2025.

[37] Daniel Richins, Dharmisha Doshi, Matthew Blackmore, Aswathy Thulaseedharan Nair, Neha Pathapati, Ankit Patel, Brainard Daguman, Daniel Dobrijalowski, Ramesh Illikkal, Kevin Long, David Zimmerman, and Vijay Janapa Reddi. AI Tax: The Hidden Cost of AI Data Center Applications. *ACM Trans. Comput. Syst.*, 37(1-4):1–32, November 2019.

[38] Hermann Schweizer, Maciej Besta, and Torsten Hoefler. Evaluating the Cost of Atomic Operations on Modern Architectures, October 2020.

[39] Debendra Das Sharma. An Introduction to the Compute Express LinkTM (CXLTM) Interconnect.

[40] Joshua Suetterlein, Joseph Manzano, and Andres Marquez. Synchronization for CXL Based Memory. In *Proceedings of the International Symposium on Memory Systems*, pages 178–185, Washington DC USA, September 2024. ACM.

[41] Wenda Tang, Ying Han, Tianxiang Ai, Guanghui Li, Bin Yu, and Xin Yang. Yggdrasil: Reducing Network I/O Tax with (CXL-Based) Distributed Shared Memory. In *Proceedings of the 53rd International Conference on Parallel Processing*, pages 597–606, Gotland Sweden, August 2024. ACM.

[42] Jesmin Jahan Tithi, Hanjiang Wu, Avishaii Abuhatzera, and Fabrizio Petrini. Scaling Intelligence: Designing Data Centers for Next-Gen Language Models, September 2025.

[43] Zuo Wang, Qi Zuo, and Jiaxing Li. An Intelligent Multi-Port Memory. In *2008 International Symposium on Intelligent Information Technology Application Workshops*, pages 251–254, December 2008.

[44] Song Wu, Jun Zhang, Yaqiong Peng, Hai Jin, and Wenbin Jiang. Optimization strategies for inter-thread synchronization overhead on NUMA machine. In *2015 IEEE 34th International Performance Computing and Communications Conference (IPCCC)*, pages 1–8, December 2015.

[45] Yiwei Yang, Brian Zhao, Yusheng Zheng, Pooneh Safayenikoo, Tanvir Ahmed Khan, and Andi Quinn. Cxlmemsim: A pure software simulated cxl.mem for performance characterization, 2025.

[46] Yuhong Zhong, Daniel S. Berger, Pantea Zardoshti, Enrique Saurez, Jacob Nelson, Antonis Psistakis, Joshua Fried, and Asaf Cidon. My CXL Pool Obviates Your PCIe Switch. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, pages 58–66, Banff AB Canada, May 2025. ACM.